



Universidade de São Paulo
Instituto de Ciências Matemáticas e de Computação

Allan Vitor de Souza Silva - 14712657
Augusto Fernandes Ildefonso - 15441810
Lucas Lombardi Castro - 13672978

Protocolo de Comunicação para Aplicação Smart: Sala de Aula Inteligente

SSC0142 - Redes de Computadores
Prof. Kalinka Regina Lucas Jaquie Castelo Branco

São Carlos, SP
2026

Sumário

1. Header do Protocolo.....	3
1.1. Definição.....	3
1.2. Estrutura.....	3
2. Mensagens e Estrutura.....	4
2.1. Sensores.....	4
2.1.1. DISCOVER (0).....	4
2.1.2. SYN (1).....	4
2.1.3. SEND_DATA (2).....	5
2.2. Atuadores.....	5
2.2.1. DISCOVER (0).....	5
2.2.2. SYN (1).....	6
2.2.3. ACK_COMMAND (3).....	6
2.3. Gerenciador.....	6
2.3.1. OFFER (4).....	6
2.3.2. ACK (5).....	7
2.3.3. ACK_DATA (6).....	7
2.3.4. SEND_COMMAND (7).....	7
2.3.5. SEND_DATA (8).....	7
2.4. Cliente.....	8
2.4.1. REQUEST_DATA (9).....	8
2.4.2. ACK (10).....	8
3. Troca de Mensagens.....	8
3.1. Sensor e Gerenciador.....	8
3.2. Atuador e Gerenciador.....	9
3.2.1. Requisito 3.2.....	10
3.2.2. Requisito 3.3.....	11
3.2.3. Requisito 3.4.....	11
3.2.4. Requisito 3.5.....	11
3.2.5. Requisito 3.6.....	12
3.2.6. Requisito 3.7.....	12
3.3. Cliente e Gerenciador (Requisito 3.8 e 4).....	12

1. Header do Protocolo

1.1. Definição

O header é um bloco de dados que é adicionado ao início de um pacote de informação antes de ser transmitido pela rede. Ele contém metadados essenciais para que os dispositivos como roteadores e servidores saibam como lidar com a mensagem.

1.2. Estrutura

O header do protocolo é um campo comum a todas as primitivas. Por isso, ele deve conter informações comuns a todas as primitivas. Antes de apresentar a estrutura do header, é preciso estabelecer que o identificador único usado para todos os dispositivos (sensor, atuador e gerenciador) foi o IPV4 em conjunto da sua porta (socket).

A estrutura do header pode ser vista a seguir:

- **TIPO_PRIMITIVA:** Tipo da primitiva da mensagem (Obrigatório, 1 byte).
- **ID_REM:** Identificador único do remetente (IP + PORTA) (Obrigatório, 6 bytes).
- **ID_DEST:** Identificador único do destinatário (IP + PORTA) (Obrigatório, 6 bytes).
- **TAM:** Quantos bytes possui a mensagem completa, payload e header (Obrigatório, 2 bytes).

O IP_DEST quando usamos o broadcast para descobrir o gerenciador é o 255.255.255.255, já nas outras requisições ele é o IP do dispositivo de destino.

Além disso, é importante ressaltar que cada tipo de primitiva foi mapeado para um número inteiro, possibilitando enviá-lo no lugar do texto, o que permite poupar bytes no cabeçalho já que com 1 byte apenas conseguimos mapear todas as mensagens. Nas mensagens a seguir está indicado qual o número que representa cada uma delas.

Usaremos com os sockets o protocolo TCP, uma vez que os dados que enviamos precisam estar intactos. Devido ao contexto do problema, uma perda de dados implicaria em uma falha na ação ou então em um aluno registrar presença mas não salvarmos ela, por exemplo. Ambos cenários são ruins e pensando no

trade-off de tempo e integridade, para o nosso projeto é importante garantir a integridade e não rapidez, por isso, usaremos TCP.

2. Mensagens e Estrutura

2.1. Sensores

Os sensores são dispositivos físicos que monitoram e enviam dados específicos de acordo com o tipo do sensor. Cada sensor tem um ID único, conforme estabelecemos anteriormente (cumpre com o requisito 1.1).

2.1.1. DISCOVER (0)

Essa primitiva é utilizada logo quando ligamos os Sensores. Eles fazem um broadcast pela rede buscando pelo Gerenciador. Como o objetivo é apenas descobrir o gerenciador, não precisamos enviar um corpo na mensagem. Por isso, seu campo é:

- **HEADER:** cabeçalho do nosso protocolo (Obrigatório, 15 bytes)

Cumpre, em parceria com o SYN, com o requisito 1.2.

2.1.2. SYN (1)

Essa primitiva é utilizada para conectar os sensores ao Gerenciador. Assim que obter a resposta do IP do gerenciador, os sensores enviam essa mensagem. A identificação do sensor para o gerenciador é feita com o ID único dele (IPV4) junto do seu tipo. Seus campos são:

- **HEADER:** cabeçalho do nosso protocolo (Obrigatório, 15 bytes)
- **TYPE:** define qual é o tipo do sensor. Os tipos possíveis foram mapeados para números inteiros: presença (0), leitor (1) e chave (2) (Obrigatório, 1 byte)

Cumpre, junto do DISCOVER, com o requisito 1.2. Além disso, como o identificador já está no cabeçalho, não precisamos enviá-lo como campo.

2.1.3. SEND_DATA (2)

Essa primitiva é usada para enviar dados do Sensor para o Gerenciador. Os sensores esperam receber uma mensagem de confirmação de recebimento dos dados. Seus campos:

- **HEADER:** cabeçalho do nosso protocolo (Obrigatório, 15 bytes)
- **DATA:** dados dos sensores (Obrigatório, tamanho variável)

O campo DATA varia de acordo com o sensor. Mas podemos mapear quais são os seus componentes de acordo com o tipo do sensor:

- Sensor de Presença de Pessoas
 - **EMPTY:** indica se há pessoas na sala, 1 se estiver vazia e 0 se tiver ao menos uma pessoa (Obrigatório, 1 byte)
 - Cumpre com o requisito 1.4
- Leitor de Cartão dos Alunos
 - **NROALUNO:** número único do aluno (Obrigatório, 7 bytes)
 - **NOME:** nome do aluno (Obrigatório, tamanho variável)
 - Cumpre com os requisitos 1.3 e 3.6.
- Chave ON/OFF do projetor
 - **STATUS:** indica o status atual, 0 significa desligado e 1 significa ligado (Obrigatório, 1 byte)
 - Cumpre com o requisito 1.5

2.2. Atuadores

Os atuadores são elementos físicos da sala, que podem executar uma função específica. Eles são controlados pelo gerenciador, seguindo os requisitos funcionais. Cada atuador tem um identificador único (ID).

2.2.1. DISCOVER (0)

Essa primitiva é enviada pelos atuadores quando são ligados. Eles então buscam o gerenciador por meio de um broadcast. Seus campos são:

- **HEADER:** cabeçalho do nosso protocolo (Obrigatório, 15 bytes)
- Cumpre, junto do SYN, com o requisito 2.2.

2.2.2. SYN (1)

Essa primitiva é usada para conectar os atuadores ao Gerenciador. Ela envia o identificador único do atuador para poder identificá-lo. Seus campos são:

- **HEADER:** cabeçalho do nosso protocolo (Obrigatório, 15 bytes)

Como o identificador já está no cabeçalho, não precisamos enviá-lo novamente como um campo.

Cumpr, junto do DISCOVER, com o requisito 2.2.

2.2.3. ACK_COMMAND (3)

Essa primitiva é enviada do atuador para o gerenciador. Ela confirma o recebimento da mensagem e o êxito do comando pedido. Seus campos são:

- **HEADER:** cabeçalho do nosso protocolo (Obrigatório, 15 bytes)
- **STATUS:** corresponde ao status da operação, 0 foi mal sucedido e 1 foi bem sucedido (Obrigatório, 1 byte)

Cumpr, em conjunto com o SEND_COMMAND do Gerenciador, com o requisito 2.3.

2.3. Gerenciador

O Gerenciador é responsável pelo controle lógico de todo o ecossistema da sala de aula. Ele é o ponto central de comunicação, conversando tanto com os sensores quanto com os atuadores. Importante ressaltar que o requisito 3.6 é garantido pelo envio dos dados pelo sensor e pela lógica interna do Gerenciador, a mensagem de ACK_DATA serve apenas para garantir a integridade da troca de mensagens.

2.3.1. OFFER (4)

Essa primitiva é enviada quando o Gerenciador recebe uma primitiva de DISCOVER, seja do sensor ou do atuador. Ela envia para o dispositivo o seu IP. Seus campos são:

- **HEADER:** cabeçalho do nosso protocolo (Obrigatório, 15 bytes)
- **ID:** identificador único, ou seja, o seu IP (Obrigatório, 4 bytes)

Cumpr com o requisito 3.1.

2.3.2. ACK (5)

Essa primitiva é enviada sempre que o Gerenciador recebe uma primitiva de SYN, seja do sensor ou do atuador. Ela indica que ele recebeu a primitiva e estabelecemos a conexão. Seus campos são:

- **HEADER:** cabeçalho do nosso protocolo (Obrigatório, 15 bytes)

Cumprir com o requisito 3.1.

2.3.3. ACK_DATA (6)

Essa primitiva é enviada para o sensor após o Gerenciador receber os dados, através da primitiva SEND_DATA. Ela serve para garantir a integridade da ação apenas. Seus campos são:

- **HEADER:** cabeçalho do nosso protocolo (Obrigatório, 15 bytes)

Cumprir com os requisitos 1.3 e 3.6.

2.3.4. SEND_COMMAND (7)

Essa primitiva é enviada pelo gerenciador aos atuadores para que eles realizem um determinado comando. Como todas as ações envolvem apenas ligar ou desligar o atuador, essa primitiva se torna comum a todos os atuadores, não sendo necessário enviar um dado específico para cada um no campo COMMAND. Seus campos são:

- **HEADER:** cabeçalho do nosso protocolo (Obrigatório, 15 bytes)
- **COMMAND:** define o estado com o atuador deve assumir, sendo 0 desligado e 1 ligado (Obrigatório, 1 byte)

Esse comando, em conjunto com a lógica de controle interna do Gerenciador e o recebimento de informações dos sensores, cumprem com os requisitos: 3.2, 3.3, 3.4, 3.5 e 3.7.

2.3.5. SEND_DATA (8)

Essa primitiva é enviada quando recebemos uma primitiva de REQUEST_DATA do cliente. Ela é usada para enviar a lista de alunos que registraram presença. Seus campos são:

- **HEADER:** cabeçalho do nosso protocolo (Obrigatório, 15 bytes)
- **DATA:** lista de tuplas no formato (número identificador, nome) dos alunos que registraram presença (Obrigatório, tamanho variável)

Cumpra com os requisitos: 3.8 e 4.

2.4. Cliente

2.4.1. REQUEST_DATA (9)

O cliente pode interagir com o gerenciador requisitando a lista de presença. Para isso, podemos criar uma só mensagem de requisição de dados. Seus campos são:

- **HEADER:** cabeçalho do nosso protocolo (Obrigatório, 15 bytes)
- **DATE:** data do dia que se quer obter a presença, no formato “dd/mm/aaaa” (Obrigatório, 10 bytes)

Cumpra com os requisitos: 3.8 e 4.

2.4.2. ACK (10)

O cliente envia essa primitiva ao receber os dados para confirmar o recebimento dos mesmos. Seus campos são:

- **HEADER:** cabeçalho do nosso protocolo (Obrigatório, 15 bytes)

Cumpra com os requisitos: 3.8 e 4.

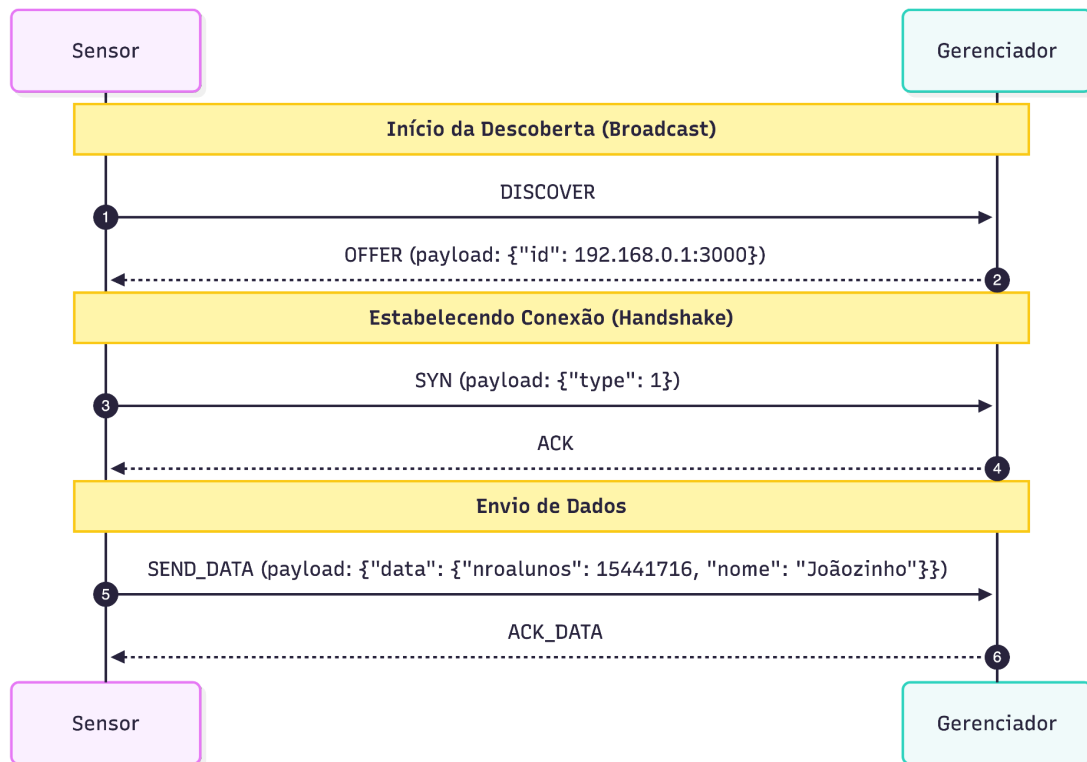
3. Troca de Mensagens

Para facilitar a visualização da troca de mensagens, omitimos alguns campos do header mantendo apenas o TIPO_PRIMITIVA e usamos o nome dela no lugar do número identificador, visando facilitar a leitura, porém nas requisições reais o header completo é enviado, assim como usamos o número de identificação e não o nome da primitiva.

3.1. Sensor e Gerenciador

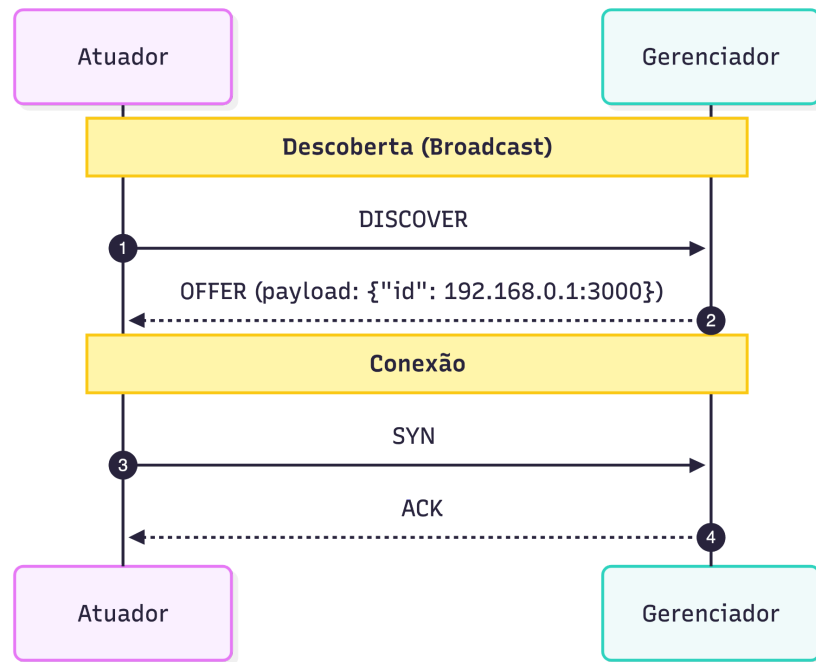
O nosso sistema possui 3 tipos de sensores e todos eles enviam o campo DATA, mudando apenas o conteúdo dentro dele. A fim de simplificar o esquema, vamos exemplificar a troca de mensagens usando o leitor de cartão como exemplo para o envio dos dados do campo DATA, mas para os outros basta apenas substituir os campos de dentro do DATA para o campo do respectivo sensor que foi estabelecido anteriormente.

Veja a seguir o diagrama dessa troca de mensagens (requisitos 1.2, 1.3, 1.4, 1.5 e 3.1):



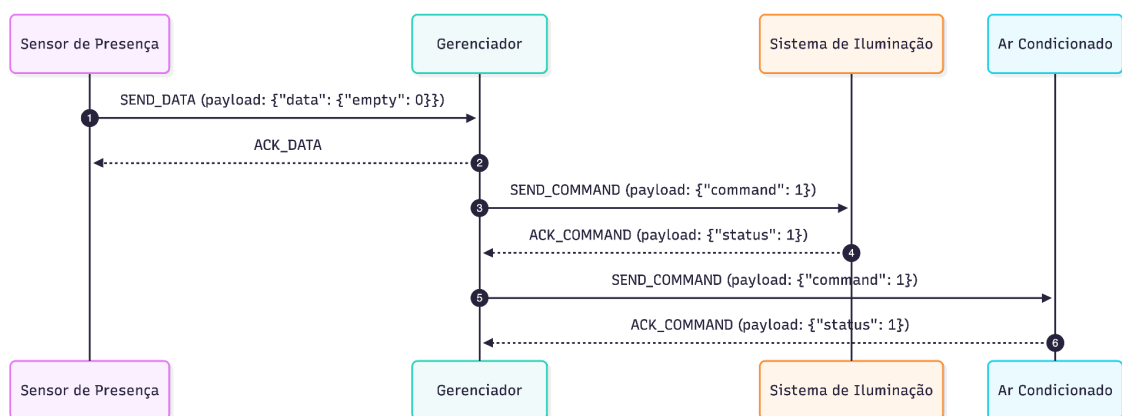
3.2. Atuador e Gerenciador

Primeiramente vamos ver o diagrama de conexão dos Atuadores com o Gerenciador (requisitos 2.2 e 3.1):

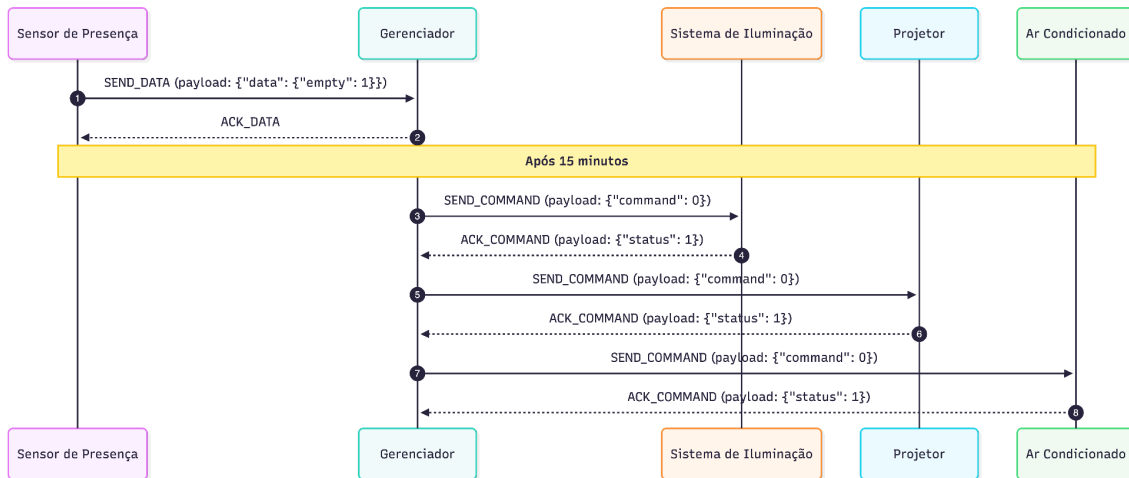


Após a fase de descoberta, exemplificada acima, o atuador, sensor e gerenciador trocaram informações, de modo a realizar os requisitos funcionais. Para os próximos diagramas vamos considerar que a descoberta e conexão de todos os componentes já foi efetuada.

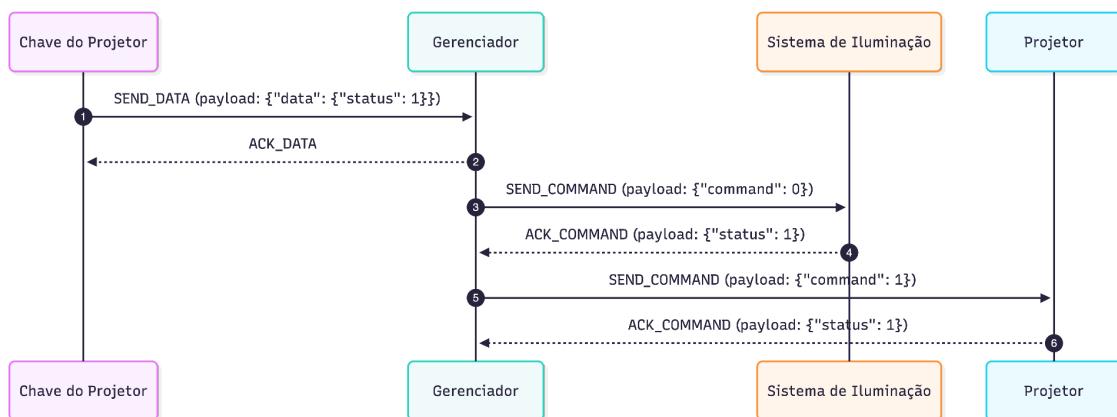
3.2.1. Requisito 3.2



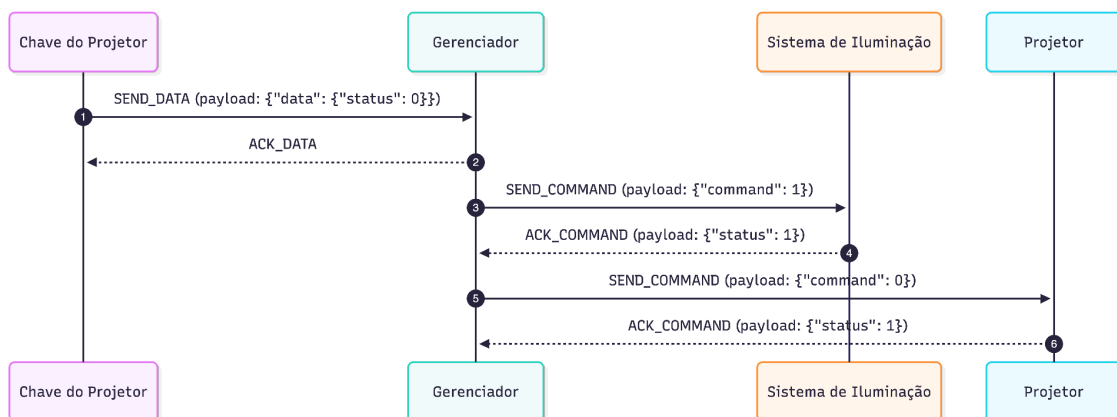
3.2.2. Requisito 3.3



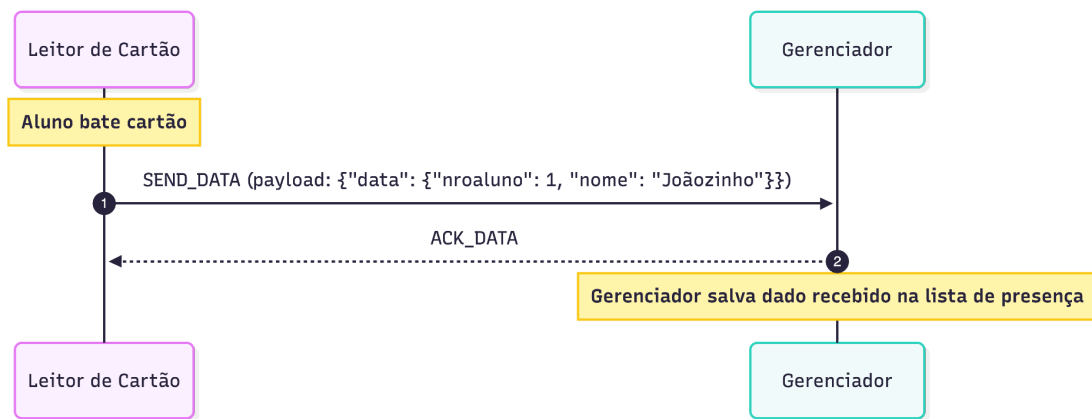
3.2.3. Requisito 3.4



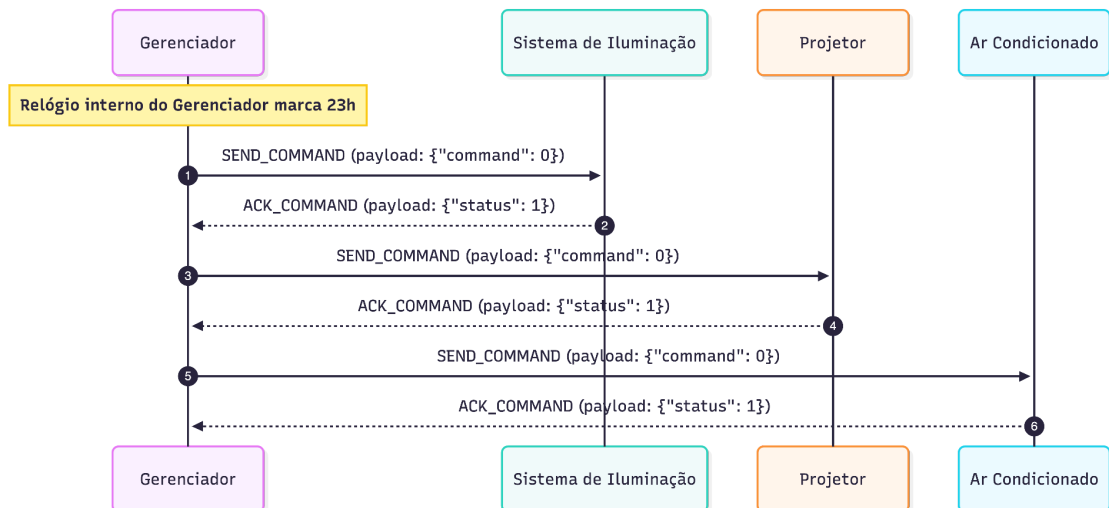
3.2.4. Requisito 3.5



3.2.5. Requisito 3.6



3.2.6. Requisito 3.7



3.3. Cliente e Gerenciador (Requisito 3.8 e 4)

Para esse diagrama temos apenas o cliente interagindo com o gerenciador, os sensores e atuadores não atuam nesse requisito (claro que leitor de cartão faz parte mas vamos considerar que ele já registrou as presenças anteriormente).

